

Taller: Complejidad Algorítmica, Arreglos 1D y 2D

Parte 1 — Complejidad algorítmica (Big O)

1.1 Clasifica la complejidad

a) **Buscar una palabra en un diccionario físico, abriendo siempre por la mitad de las páginas que quedan.**

- **Complejidad:** $O(\log n)$
 - **Justificación:** Cada vez que abres el diccionario por la mitad, descartas la mitad de las páginas restantes. Esto significa que el número de páginas que debes revisar se reduce a la mitad en cada paso. Si tienes n páginas, después del primer paso te quedan $n/2$, luego $n/4$, luego $n/8$, y así sucesivamente. El número de pasos necesarios para llegar a una sola página es proporcional al logaritmo en base 2 de n . Por eso es $O(\log n)$.
-

b) **Revisar, uno por uno, cada carné en una caja de carnés de estudiantes hasta encontrar el que buscas.**

- **Complejidad:** $O(n)$
 - **Justificación:** En el peor caso, podrías tener que revisar todos los carnés de la caja antes de encontrar el que buscas. Si hay n carnés, en el peor caso haces n comparaciones. El tiempo crece linealmente con el número de carnés, por eso es $O(n)$.
-

c) **Sacar la primera carta de un mazo ya barajado.**

- **Complejidad:** $O(1)$
 - **Justificación:** No importa cuántas cartas tenga el mazo, sacar la primera carta siempre requiere una sola operación: tomar la carta que está arriba. El tiempo no depende del tamaño del mazo, es constante.
-

d) Comparar cada estudiante de un salón con cada uno de los demás estudiantes, para ver si algún par cumple años el mismo día.

- **Complejidad:** $O(n^2)$
 - **Justificación:** Si hay n estudiantes, el primero se compara con $n-1$ estudiantes, el segundo con $n-2$ (porque ya se comparó con el primero), y así sucesivamente. El número total de comparaciones es aproximadamente $n \times (n-1) / 2$, que crece proporcionalmente a n^2 . Por eso es $O(n^2)$.
-

e) Consultar la hora en tu reloj.

- **Complejidad:** $O(1)$
 - **Justificación:** Mirar el reloj es una operación instantánea que no depende de ningún tamaño de entrada. Siempre toma el mismo tiempo, sin importar el contexto, por eso es $O(1)$.
-

1.2 De la vida real al análisis

Tarea cotidiana $O(n)$:

- **Descripción:** Contar monedas de una alcancía una por una para saber cuánto dinero hay. Si tienes n monedas, debes revisar cada una individualmente para sumar su valor.
 - **Si la entrada se hace 10 veces más grande:** Si originalmente tenías 50 monedas y ahora tienes 500 (10 veces más), el tiempo que tardas en contarlas también se multiplica por 10, porque debes revisar cada moneda individualmente. Esto es característico de $O(n)$: el tiempo escala linealmente con el tamaño de la entrada.
-

1.3 ¿Cuál escalar mejor?

Forma A: $O(n^2)$

Forma B: $O(n \log n)$

- **Recomendación:** Usaría la **Forma B** ($O(n \log n)$) si no sé qué tan grande será n en el futuro.
- **Justificación:** Aunque para $n = 5$ la Forma A sea más rápida en la práctica (por constantes pequeñas o menor sobrecarga), a medida que n crece, n^2 crece mucho más rápido que $n \log n$. Por ejemplo:
 - Si $n = 1000$: $n^2 = 1,000,000$ vs $n \log n \approx 10,000$

- Si $n = 1,000,000$: $n^2 = 1,000,000,000,000$ vs $n \log n \approx 20,000,000$
 - La diferencia se vuelve abismal. Big O describe el comportamiento asintótico (cuando n tiende a infinito), y $O(n \log n)$ escala mucho mejor que $O(n^2)$.
-

1.4 Verdadero o falso (justifica siempre)

a) Un algoritmo $O(1)$ siempre es más rápido en segundos reales que uno $O(n)$.

- **Falso.**
 - **Justificación:** Big O describe cómo crece el tiempo de ejecución a medida que crece la entrada, no el tiempo absoluto en segundos. Un algoritmo $O(1)$ podría tener una constante muy grande (por ejemplo, 1000 segundos fijos), mientras que un algoritmo $O(n)$ podría tener una constante muy pequeña (por ejemplo, 0.001 segundos por elemento). Para n pequeño, el $O(n)$ podría ser más rápido en la práctica.
-

b) La notación Big O describe cómo crece el tiempo de ejecución a medida que crecen los datos, no el tiempo exacto en segundos.

- **Verdadero.**
 - **Justificación:** Big O es una notación asintótica que captura la tasa de crecimiento del tiempo (o espacio) en función del tamaño de la entrada n . Ignora constantes y términos de menor orden, enfocándose en el comportamiento cuando n es muy grande.
-

c) Buscar por índice en un arreglo (arreglo) es $O(n)$, porque hay que recorrer los primeros 5 elementos.

- **Falso.**
 - **Justificación:** Acceder a un elemento por índice en un arreglo es $O(1)$ porque la dirección de memoria se calcula directamente con la fórmula $\text{dirección} = \text{base} + (\text{índice} \times \text{tamaño})$. No hay que recorrer elementos previos; el acceso es directo e instantáneo, sin importar el índice.
-

Parte 2 — Arreglos 1D

2.1 Diseña el arreglo

Arreglo **notas** con 6 casillas (índices desde 0):



- **Tercera nota ingresada:**
 - **Valor:** 2.8
 - **Índice:** 2 (porque los índices empiezan en 0: primera = índice 0, segunda = índice 1, tercera = índice 2).
- **Instrucción en pseudocódigo para acceder a la última nota sin recorrer el arreglo:**
text
- `ultimaNota = notas[6]`
- **Justificación:** Sabemos que el arreglo tiene 6 elementos, así que el último índice es 5 (tamaño - 1). El acceso por índice es 0(1).

2.2 Direcciones de memoria

Datos:

- Dirección base: 0x2000
- Tamaño por valor: 4 bytes
- Fórmula: $\text{dirección} = \text{base} + (\text{índice} \times \text{tamaño})$

Cálculos:

- **notas:**
 $\text{dirección} = 0x2000 + (0 \times 4) = 0x2000 + 0 = 0x2000$

notas:

$\text{dirección} = 0x2000 + (3 \times 4) = 0x2000 + 12 = 0x200C$

notas:

$\text{dirección} = 0x2000 + (5 \times 4) = 0x2000 + 20 = 0x2014$

¿Por qué acceder a notas es igual de rápido que acceder a notas?

- **Justificación:** Porque la dirección de memoria se calcula directamente con la fórmula $\text{base} + (\text{índice} \times \text{tamaño})$. No importa qué índice pidas, el cálculo es una sola operación aritmética (multiplicación y suma). No hay que recorrer elementos previos, por eso el acceso es $O(1)$ para cualquier índice.
-

2.3 Búsqueda lineal vs. binaria

Arreglo ordenado: $\text{edades} = [15, 18, 20, 23, 27, 31, 35, 40]$

Valor a buscar: 31

Búsqueda lineal (paso a paso):

1. Comparar 15 con 31 → no es
 2. Comparar 18 con 31 → no es
 3. Comparar 20 con 31 → no es
 4. Comparar 23 con 31 → no es
 5. Comparar 27 con 31 → no es
 6. Comparar 31 con 31 → **encontrado**
- **Número de comparaciones:** 6
 - **Justificación:** La búsqueda lineal revisa elemento por elemento desde el inicio hasta encontrar el valor o llegar al final. En el peor caso, hace n comparaciones ($O(n)$).
-

Búsqueda binaria (paso a paso):

- **Inicial:** $\text{primero} = 0, \text{ultimo} = 7$ (8 elementos, índices 0 a 7)
1. **Paso 1:**
 - $\text{medio} = (0 + 7) / 2 = 3$ (índice 3 → valor 23)
 - Comparar 23 con 31 → $23 < 31$, buscar en la mitad derecha
 - $\text{primero} = 4, \text{ultimo} = 7$
 2. **Paso 2:**
 - $\text{medio} = (4 + 7) / 2 = 5$ (índice 5 → valor 31)
 - Comparar 31 con 31 → **encontrado**

- **Número de comparaciones:** 2
 - **Justificación:** La búsqueda binaria divide el arreglo por la mitad en cada paso, descartando la mitad que no puede contener el valor. Para $n = 8$, $\log_2(8) = 3$, y en este caso encontramos en 2 comparaciones (mejor que el peor caso).
-

¿Por qué la búsqueda binaria solo funciona si el arreglo ya está ordenado?

- **Justificación:** La búsqueda binaria decide en cada paso si buscar en la mitad izquierda o derecha comparando el valor del medio con el objetivo. Esta decisión solo es válida si sabemos que todos los valores a la izquierda son menores (o iguales) y todos los de la derecha son mayores (o iguales). Si el arreglo no está ordenado, no podemos garantizar que el valor buscado esté en una u otra mitad, y el algoritmo fallaría.
-

Si el arreglo tuviera 1 millón de elementos, ¿cuál búsqueda usarías?

- **Respuesta:** Búsqueda binaria ($O(\log n)$).
 - **Justificación:**
 - Búsqueda lineal: en el peor caso, 1,000,000 comparaciones ($O(n)$).
 - Búsqueda binaria: en el peor caso, $\log_2(1,000,000) \approx 20$ comparaciones ($O(\log n)$).
 - La diferencia es abismal. Para grandes volúmenes de datos, $O(\log n)$ es mucho más eficiente que $O(n)$.
-

2.4 Insertar un elemento

Arreglo: `edades = [15, 18, 20, 23, 27]` (5 casillas, sin espacio libre)

Insertar al final:

- **Si hay espacio disponible:**
 - **Complejidad:** $O(1)$
 - **Justificación:** Solo hay que escribir el nuevo valor en la siguiente casilla libre. Es una operación directa, sin mover elementos.
- **Si no hay espacio y hay que crear un arreglo más grande:**
 - **Complejidad:** $O(n)$
 - **Justificación:** Hay que crear un nuevo arreglo de mayor tamaño, copiar todos los n elementos del arreglo viejo al nuevo, y luego agregar el nuevo elemento. Copiar n elementos toma tiempo proporcional a n .

Insertar el valor 21 en la mitad (para que quede ordenado):

- **Posición correcta:** Entre 20 (índice 2) y 23 (índice 3), es decir, en el índice 3.
- **Qué hay que hacer:**
 1. Mover 27 una posición a la derecha (de índice 4 a índice 5)
 2. Mover 23 una posición a la derecha (de índice 3 a índice 4)
 3. Escribir 21 en el índice 3
- **Complejidad:** $O(n)$
- **Justificación:** En el peor caso (insertar al inicio), hay que mover todos los n elementos una posición a la derecha para hacer espacio. Incluso en el mejor caso (insertar al final), si hay que mover elementos, el tiempo es proporcional al número de elementos desplazados.

Comparación en Big O:

Caso	Complejidad	Justificación
Insertar al final (hay espacio)	$O(1)$	Escritura directa, sin mover elementos
Insertar al final (no hay espacio)	$O(n)$	Copiar todos los elementos a un nuevo arreglo
Insertar en medio (ordenado)	$O(n)$	Mover elementos para hacer espacio

Parte 3 — Arreglos 2D

3.1 Diseña la matriz

Matriz **salon** de 3 filas × 4 columnas:

	Columna 0	Columna 1	Columna 2	Columna 3
Fila 0	[0][0]	[0][1]	[0][2]	[0][3]
Fila 1	[1][0]	[1][1]	[1][2]	[1][3]
Fila 2	[2][0]	[2][1]	[2][2]	[2][3]

Ejemplo de contenido (1 = ocupado, 0 = libre):

	Col 0	Col 1	Col 2	Col 3
Fila 0	1	0	1	1
Fila 1	0	1	0	1
Fila 2	1	1	0	0

Pseudocódigo para acceder al puesto de la fila 2, columna 3:

	Col 0	Col 1	Col 2	Col 3
Fila 0	[0][0] Índice 0 0x1000 Esquina	[0][1] Índice 1 0x1004 Borde	[0][2] Índice 2 0x1008 Borde	[0][3] Índice 3 0x100C Esquina
Fila 1	[1][0] Índice 4 0x1010 Borde	[1][1] Índice 5 0x1014 Interior	[1][2] Índice 6 0x1018 Interior	[1][3] Índice 7 0x101C Borde
Fila 2	[2][0] Índice 8 0x1020 Esquina	[2][1] Índice 9 0x1024 Borde	[2][2] Índice 10 0x1028 Borde	[2][3] Índice 11 0x102C Esquina Elemento accedido
Clasificación	Esquina — 4 celdas (0,0) (0,3) (2,0) (2,3)	Borde — 6 celdas (0,1) (0,2) (1,0) (1,3) (2,1) (2,2)	Interior — 2 celdas (1,1) (1,2)	

Justificación: En una matriz, se accede con `[fila][columna]`. La fila 2 es la tercera fila (índices desde 0), y la columna 3 es la cuarta columna.

¿Cuántas casillas tiene en total esta matriz?

- **Total:** $3 \times 4 = 12$ casillas
- **Fórmula general:** $total = filas \times columnas$
- **Justificación:** Cada fila tiene $columnas$ casillas, y hay $filas$ filas, así que el producto da el número total de elementos.

3.2 De 2D a memoria (row-major)

Datos:

- Dirección base: $0x1000$
- Tamaño por valor: 4 bytes
- Orden: row-major (fila por fila)

Matriz aplanada en memoria :

	Col 0	Col 1	Col 2	Col 3
Fila 0	[0][0] Índice 0 0x1000 Esquina	[0][1] Índice 1 0x1004 Borde	[0][2] Índice 2 0x1008 Borde	[0][3] Índice 3 0x100C Esquina
Fila 1	[1][0] Índice 4 0x1010 Borde	[1][1] Índice 5 0x1014 Interior	[1][2] Índice 6 0x1018 Interior	[1][3] Índice 7 0x101C Borde
Fila 2	[2][0] Índice 8 0x1020 Esquina	[2][1] Índice 9 0x1024 Borde	[2][2] Índice 10 0x1028 Borde	[2][3] Índice 11 0x102C Esquina
Clasificación	Esquina — 4 celdas Exposición alta	Borde — 6 celdas Exposición media	Interior — 2 celdas Exposición baja	

Fórmula para dirección en row-major:

$\text{dirección} = \text{base} + ((\text{fila} \times \text{número de columnas} + \text{columna}) \times \text{tamaño} \sim o)$

Cálculo para:

- $\text{fila} = 1, \text{columna} = 2, \text{columnas} = 4, \text{tamaño} = 4$

$\text{dirección} = 0x1000 + ((1 \times 4 + 2) \times 4) = 0x1000 + (6 \times 4) = 0x1000 + 24 = 0x1018$

Cálculo para :

- $\text{fila} = 0, \text{columna} = 0$

$\text{dirección} = 0x1000 + ((0 \times 4 + 0) \times 4) = 0x1000 + 0 = 0x1000$

Cálculo para (última casilla):

- $\text{fila} = 2, \text{columna} = 3$

$$\text{dirección} = 0 \times 1000 + ((2 \times 4 + 3) \times 4) = 0 \times 1000 + (11 \times 4) = 0 \times 1000 + 44 = 0 \times 102C$$

3.3 Recorrido con ciclos anidados

Pseudocódigo para contar puestos ocupados (1):

```
contador = 0
para fila desde 0 hasta 2:
    para columna desde 0 hasta 3:
        si salon[fila][columna] == 1:
            contador = contador + 1
```

O en forma general para f filas y c columnas:

```
contador = 0
para i desde 0 hasta f-1:
    para j desde 0 hasta c-1:
        si salon[i][j] == 1:
            contador = contador + 1
```

Complejidad en Big O:

- **Complejidad:** $O(f \times c)$ o $O(n)$ donde $n = f \times c$ (número total de casillas)
 - **Justificación:** El ciclo externo se ejecuta f veces, y por cada iteración del externo, el ciclo interno se ejecuta c veces. En total, se visitan $f \times c$ casillas, una por una. Por eso la complejidad es proporcional al número total de elementos.
-

¿Qué pasa si f y c se duplican ambos a la vez?

- Si originalmente f filas y c columnas $\rightarrow f \times c$ casillas
 - Si se duplican: $2f$ filas y $2c$ columnas $\rightarrow (2f) \times (2c) = 4 \times (f \times c)$ casillas
 - **Consecuencia:** El tiempo de ejecución se multiplica por 4, porque hay 4 veces más casillas que recorrer. Esto es consistente con $O(f \times c)$: si ambas dimensiones se duplican, el total de elementos se cuadruplica.
-

3.4 Conectando todo

Relación entre Big O, arreglos 1D y arreglos 2D:

Entender la complejidad (Big O) te ayuda a predecir cómo escalará el tiempo de ejecución de un algoritmo a medida que crecen los datos. En arreglos 1D, acceder por índice es $O(1)$ porque la dirección se calcula directamente, pero buscar linealmente es $O(n)$ porque puede requerir revisar todos los elementos. En arreglos 2D, recorrer toda la matriz con ciclos anidados es $O(f \times c)$ porque visitamos cada casilla una vez. Saber esto te permite decidir: si necesitas acceso rápido por posición, un arreglo (1D o 2D) es ideal; si necesitas buscar un valor, una búsqueda binaria ($O(\log n)$) es mejor que una lineal ($O(n)$) pero solo si los datos están ordenados. En general, Big O te guía para elegir la estructura y el algoritmo más eficientes según el tamaño esperado de los datos.